

Hypervisor vCPU 调度策略和配置方法

GRTNBL202002XXXX

版本号，日期

公开 或 内部使用



变更履历表

版本	日期	修订人	修订内容
0.1	2024-11-08	陈平平	初稿

目录

- Hypervisor vCPU 调度策略和配置方法 1
- 1 Hypervisor VCPU 调度方案 6
 - 1.1 虚拟机 CPU 配置 6
 - 1.2 Nebula CPU 调度策略 7
 - 1.2.1 Nebula 对共享核调度策略 7
 - 1.2.2 Nebula 共享核调度策略参数配置 8

1 Hypervisor VCPU 调度方案

1.1 虚拟机 CPU 配置

对于 MT8676 或 MT8678 这颗 SOC 来说，它有 8 个物理核。其中有 4 个小核，分别是 Core0~Core3。有 3 个中核，分别是 Core4~Core6。有 1 个大核，是 Core7。对于每一个物理 CPU，在 hypervisor 中，都有一个虚拟的 vCPU 与之对应。

物理 CPU 核既可以让一个 VM 独享，也可以让几个 VM 共享。如果 CPU 为某一个 VM 独享，需要在除该 VM 之外的所有其他 VM 的配置文件中，将该 CPU 设置成 offline。如果要多个 VM 共享 CPU，则要在相应的 VM 配置文件中，让该 CPU 处于 onLine。

➤ MT8676 设置 SOS CPU offline 的方法如下：

在编译时修改

在文件 `yocto/prebuilt/hypervisor/grt_mt8676/nbl_vm_pre.sh` 中，添加
`echo 0 > /sys/devices/system/cpu/cpux/online //cpux → cpu0, cpu1, cpu2, ... cpu7`
 对于 SOS 来说，Core0 用于接收中断，不能关闭。该方法是编译时生效。

在系统运行时修改

通过 SOS adb shell 进入目录 `/vendor/etc/hyper_android`，修改 `nbl_vm_pre.sh` 文件，保存重启。

➤ MT8676 设置 UOS (Android) CPU offline 的方法如下：

在编译时修改

在文件 `alps/devices/mediatek/mt6897/init.mt6897.rc` 中，添加
`write /sys/devices/system/cpu/cpux/online 0 //cpu x → cpu0, cpu1, cpu2, ... cpu7`
 对于 UOS (Android)，Core2 用于接收中断，不能设置成 offline。

在运行时修改

通过 Android adb shell 进入目录 `/vendor/etc/init/hw`，修改 `init.mt8676.rc` 文件，保存重启。

➤ MT8678 设置 SOS CPU offline 的方法如下：

在编译时修改

在文件 `yocto/prebuilt/hypervisor/grt/nbl_yocto_cpu_offline.sh` 中，设置 cpu 的 offline 状态。

```

result=$(/usr/bin/secure_reboot -ping)
while true; do
    output=$(/usr/bin/secure_reboot -ping)
    if echo "$output" | grep -q "Android : Online"; then
        echo "Android is Online"
        echo 0 > /sys/devices/system/cpu/cpu2/online
        echo 0 > /sys/devices/system/cpu/cpu3/online
        echo 0 > /sys/devices/system/cpu/cpu5/online
        echo 0 > /sys/devices/system/cpu/cpu6/online
        echo 0 > /sys/devices/system/cpu/cpu7/online
        break
    else
        echo "Android is not Online, sleeping for 2 second"
        sleep 1
    fi
done

```

图示 1: 设置 CPU offline

在运行时修改

通过 S0S adb shell 进目录 /vendor/etc/hyper_android/ 目录，修改文件 nbl_yocto_cpu_offline.sh，保存重启。

76 上面设置 CPU offline 的文件，后面会对齐 78 和 78 保持一致。

➤ MT8678 设置 UOS(Android) CPU offline 的方法如下：

在编译时修改

在文件 alps/device/mediatek/mt6991/init.mt6991.rc 文件中，添加
write /sys/devices/system/cpu/cpux/online 0 // 将 cpux 设置为 offline。

在运行时修改

通过 UOS adb shell 进目录 /vendor/etc/init/hw/init.mt6991.rc 文件中，添加
write /sys/devices/system/cpu/cpux/online 0 // 将 cpux 设置为 offline。保存重启。

对于 tbox 的 CPU 设置，后面补充。

1.2 Nebula CPU 调度策略

1.2.1 Nebula 对共享核调度策略

对于 VM 独享的 CPU，nebula 不参与 VM 内部进程调度。由 VM 的 Linux 调度程序来调度。

对于 VM 共享的 CPU，nebula 参与 VM 间的进程调度。调度策略为：

1. 将共享的 CPU 运行时间，以 period 为单位进行划分。让每个 VM 中的进程都有机会在 period 时间内运行，不会让一个 VM 独占一个 period 时间。
2. 在 period 时间单位的基础上，规定每个 VM 在 period 的运行时间 budget。即每个 VM 在 period 内最多只能运行的时间为 budget，时间到了之后，就必须让出 CPU，让其他的 VM 进行运行。
3. 对 VM 内部的进程，nebula 将其分 4 类，分别为 IRQ，Spin_Lock，RT_Thread 和

Normal_Thread。这四种进程的默认等级优先级为 $IRQ > Spin_Lock = RT_Thread > Normal_Thread$ 。

4. 对于相同优先级的进程，同时要在共享核上运行，则 nebula 会通过调度，让其交替运行。这四类进程的时间片分别是：IRQ(100us)，Spin_Lock(200us)，RT_Thread(200us)，Normal_Thread(budget)。例如，如果 SOS 和 UOS 上同时有两个 RT_Thread 进程要在相同的 CPU 上运行，则 RT_Thread(SOS) 和 RT_Thread(UOS) 交替 200us 运行。

1.2.2 Nebula 共享核调度策略参数配置

➤ 设置共享核 period 的方法：

在文件 `yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos_mt8676.json` 中，“period”字段对应的值为 period 时间，单位 us。8 个核，所以有 8 个值。

➤ 设置共享核 budget 的方法：

SOS 设置在文件 `yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos_mt8676.json` 中，“budgets”字段对应的值为 budgets 时间，单位 us。8 个核，所有有 8 个值。

UOS 设置在文件 `yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/uos_alps_pv8676.lua` 中，budgets 数组中的值为 budgets 时间，单位 us。8 个核，所有有 8 个值。

➤ 设置 RT_Thread, Spin_Lock 和 IRQ 的优先级和时间片。

在文件 `yocto/src/hypervisor/grt/thy-sdk/products/mt8676-mix/guest-configs/sos_mt8676.json` 中。“sched-priority”对应的字段为相对于 Normal_Thread 的优先级，分别对应 RT_Thread, Spin_Lock 和 IRQ 进程优先级。“sched-timeslice”对应的字段为 RT_Thread, Spin_Lock 和 IRQ 的时间片，单位 us。